DAY 3

```
#include <iostream>
#include <vector>
using namespace std;

// 1. SELECTION SORT
// Time Complexity: O(n^2)
// Space Complexity: O(1)

void selectionSort(vector<int>& arr) {
    int n = arr.size();

    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;

        // Find the minimum element in the unsorted part
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }

        // Place minimum element at the correct position
        swap(arr[i], arr[minIndex]);
    }
}

// 2. INSERTION SORT
// Time Complexity: O(n^2) worst case
// Space Complexity: O(1)

void insertionSort(vector<int>& arr) {
    int n = arr.size();

    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;

        // Shift elements greater than key to the right
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }

        // Insert key at its correct position
        arr[j + 1] = key;
```

```
    }
}

// 3. QUICK SORT
// Average Time Complexity: O(n log n)
// Worst-case Time Complexity: O(n^2)
// Space: O(log n) average recursion stack

int partition(vector<int>& arr, int low, int high) {
    // Choose the last element as pivot
    int pivot = arr[high];

    // i marks the end of elements smaller than or equal to pivot
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }

    // Put pivot at its correct sorted position
    swap(arr[i + 1], arr[high]);

    return i + 1;
}

void quickSort(vector<int>& arr, int low, int high) {
    if (low < high) {
        // Partition the array around the pivot
        int pivotIndex = partition(arr, low, high);

        // Sort left and right parts recursively
        quickSort(arr, low, pivotIndex - 1);
        quickSort(arr, pivotIndex + 1, high);
    }
}

// Function to display the array
void display(vector<int>& arr) {
    for (int x : arr) {
        cout << x << " ";
    }
    cout << endl;
}

int main() {
```

```
int n;

cout << "Enter number of elements: ";
cin >> n;

vector<int> arr(n);

cout << "Enter array elements: ";
for (int i = 0; i < n; i++) {
    cin >> arr[i];
}

int choice;

cout << "\nChoose Sorting Algorithm:\n";
cout << "1. Selection Sort\n";
cout << "2. Insertion Sort\n";
cout << "3. Quick Sort\n";
cout << "Enter your choice: ";
cin >> choice;

cout << "\nOriginal Array: ";
display(arr);

switch (choice) {
    case 1:
        selectionSort(arr);
        cout << "After Selection Sort: ";
        break;

    case 2:
        insertionSort(arr);
        cout << "After Insertion Sort: ";
        break;

    case 3:
        quickSort(arr, 0, n - 1);
        cout << "After Quick Sort: ";
        break;

    default:
        cout << "Invalid choice!" << endl;
        return 0;
}

display(arr);

return 0;
```

}

NOTES

⇒ Sorting algos

1. Selection sort → TC ⇒ $O(n^2)$ (in avg, best & worst case), SC ⇒ $O(1)$

↳ Select minimum value in each iteration
↳ after k iterations we get k smallest elements of the array.

* In-place algo
* not-stable because it can change the relative order of equal elements in the final sorted array.

eg:- arr = [64, 25, 12, 22, 11] (indices 0 1 2 3 4)

at 0th index → min(64, 25, 12, 22, 11)
= 11 ∴ swap(arr[0], arr[4])
↳ at 4th index

at 1st index → min(25, 12, 22, 64)
= 12 ∴ swap(arr[1], arr[2])

2. Insertion Sort → TC: $O(n^2)$, SC: $O(1)$

TC → O(n) in best case

↳ Take an element and place it at the correct position

* In-place
* stable

eg:- arr = [23, 1, 10, 5, 4]

1st iteration → [23], 1, 10, 5, 4 → 23 at correct position for array of size 1
2nd → [23, 1], 10, 5, 4 → 1 not at the correct position for array of size 2 → (23, 1) ∴ swap
3rd → [1, 23, 10], 5, 4
4th → [1, 10, 23, 5], 4
5th → [1, 5, 10, 23, 4]
→ final array: 1, 4, 5, 10, 23

3. Quick Sort → TC: O(n log n),

① Choose a pivot

* Not stable

① Choose a pivot
② All elements on the right of the pivot should be greater and those on the left shoul be smaller
③ This is done recursively

* Not stable
* In place

eg:- a = [4,6,2,5,7,9,1,3]
0 1 2 3 4 5 6 7

first element as pivot

quickSort(a,0,7)
↓
pivot = partition(a,0,7)
quicksort(a,0,6)
quick Sort(a,7,7)

no further calls ∵ violates low < high condn

again calls

quickSort(a,0,5)
quick sort(a,5,6)
↓
and so on...

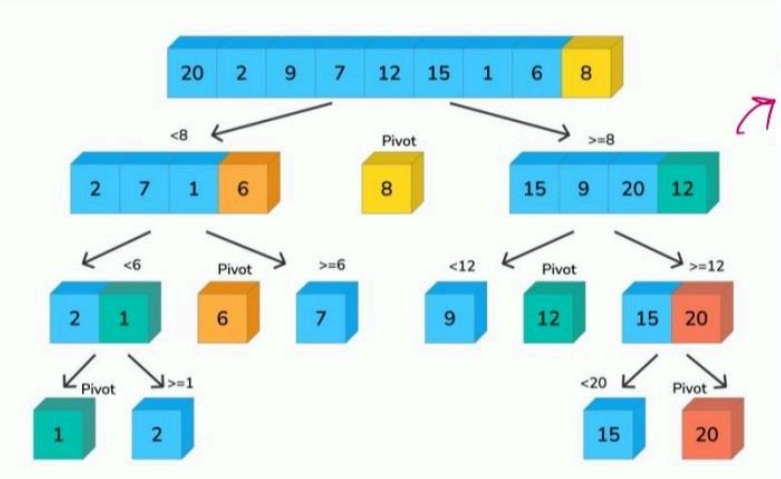
4, 6, 2, 5, 7, 9, 1, 3 6

swap

also swap pivot & a[j]

6, 3, 2, 5, 7, 9, 1, 4

returns 7



Quick Sort example with last element as pivot